



Deep learning

II Redes Neuronales: Perceptrón Multicapa para clasificación MLP

Tarea 2

Detección de fraudes en reclamación de seguros

Utilice el Notebook de Jupyter de la tarea anterior (SVM) y realice lo siguiente:

1. Agregue un título distintivo con *Markdown*
<h1>Perceptrón Multicapa</h1>
2. Haga 1 primer modelo de red neuronal con 1 sola capa y un número de neuronas entre 10 y 100, por ejemplo:

```
from sklearn.neural_network import MLPClassifier
nn = MLPClassifier(
    activation=('relu'),
    hidden_layer_sizes=70,
    alpha=0.001,
    random_state=20,
    early_stopping=False
)
# Train the model
nn.fit(X_train, y_train)
```

3. Obtenga el porcentaje de exactitud y muestre la matriz de confusión del modelo.
4. Realice una búsqueda de la mejor configuración del modelo con GridSearchCV
 - usando al menos 3 valores para:
'hidden_layer_sizes': [30, 50, 70, 90, 110, (20,20), (40,40), (60,60), (80,80), (100,100)],
 - Usando al menos 3 valores para:
'learning_rate_init': [0.0001, 0.001, 0.01, 0.1, 1, 10, 100],
 - Usando al menos 2 funciones de activación:
'activation': ['logistic', 'tanh', 'relu']
 - Use el estimador:
estimator = MLPClassifier(solver = 'lbfgs', random_state = 123)

Use todo el conjunto X ya que GridSearchCV internamente divide en datos de entrenamiento y prueba.



5. Muestre los mejores 15 o 20 resultados ordenándolos por 'mean_test_score'.
6. Obtenga el porcentaje de exactitud y muestre la matriz de confusión del mejor modelo obtenido.
7. Muestre los pesos y el bias del modelo.
8. Usando Keras genere una red neuronal con 1 capa oculta que tenga entre 10 y 70 neuronas y la capa de salida que tenga 1 sola neurona.
Puede elegir como función de activación: relu, tanh, softmax o sigmoid.
Por ejemplo:

```
import numpy as np
from keras.models import Sequential
from keras.layers import LSTM,Dense

model = Sequential()
model.add(Dense(50, input_dim=98, activation='tanh'))
model.add(Dense(1, activation='relu'))
```

Para revisar en detalle las funciones de activación consulte:
<https://keras.io/api/layers/activations/>

Recuerde que la ventaja de Keras es que permite configurar una función de activación diferente en cada capa y que permite paralelizar ciertos procesos del modelo si cuenta con tarjeta gráfica Cuda.
9. Obtenga el porcentaje de exactitud y muestre la matriz de confusión del modelo.

***NOTA 1:** Puede suceder que la matriz de confusión tenga 1 columna de ceros, esto indicaría que el modelo **no sirve** pues siempre da el mismo resultado sin importar los datos, esto suele ocurrir cuando los datos están desbalanceados como en este caso (75% no fraude-25% fraude). Si sucede esto genere otro modelo.

***NOTA 2:** Puede suceder que el modelo genere categorías adicionales, esto sucede cuando los datos de prueba son muy diferentes a los de entrenamiento o presentan outliers o hay que afinar más el modelo (aumentar o reducir número de neuronas o de capas).
10. Repita los pasos 8 y 9 pero con otra configuración.
11. Realice el diagrama del mejor modelo de red neuronal con viznet.



12. En un Excel que adjunte o una tabla html de jupyter-lab registre los mejores resultados de cada configuración.

Modelo	Configuración	% exactitud	# fraudes detectados Categoría 1
Mejor modelo SVM	Kernel: C: Gamma:		
Mejor red neuronal scikit-learn	Función activación: , learning rate: Capa oculta 1: # neuronas		
Mejor red neuronal Keras	Capa oculta 1: # neuronas, función Capa salida: 1 neurona, función		

Como podrá notar SVM obtuvo un % exactitud mayor que los modelos de redes neuronales que difícilmente superan el 72% sin embargo recuerde que el objetivo del problema es detectar los fraudes, aunque ello suponga más falsos positivos (el modelo indica que es fraude cuando no lo es). Si lo ve desde el punto de vista de la empresa, es mejor hacer unas investigaciones extra (falsos positivos) que dejar pasar más fraudes porque el modelo no los detectó (falsos negativos).



El problema que se presenta aquí es porque los datos están desbalanceados (75% no fraude, 25% fraude) lo que provoca el error mencionado en el paso 9 ya que el modelo trata de obtener una mayor exactitud menospreciando o ignorando la clase 1 (fraude). Esto nos lleva al siguiente tema de machine learning “Mejora en modelos de machine learning con datos no balanceados”.